## Project Coding – Frontend source (Next.js / TypeScript)

Seminar Hall Booking System – representative Next.js/TS sources (full repo on GitHub)

**Full source code** – Included representative code segments only (routing, auth, halls, requests, coordinator workflow). Admin/management handlers and other pages follow the same patterns. Repository: https://github.com/sreejithr247/seminar-hall-booking-system.

**frontend/app/layout.tsx**

```
import type { Metadata } from "next";
import { Inter } from "next/font/google";
import "./globals.css";
import { Toaster } from "sonner";
import { AuthProvider } from "@/lib/auth-context";

const inter = Inter({ subsets: ["latin"] });

export const metadata: Metadata = {
  title: "Seminar Hall Booking System",
  description: "IGNOU BCA BCSP-064 - Seminar Hall Booking System",
};

export default function RootLayout({
  children,
}: Readonly<{
  children: React.ReactNode;
}>) {
  return (
    <html lang="en">
      <body className={inter.className}>
        <AuthProvider>
          {children}
          <Toaster position="bottom-right" />
        </AuthProvider>
      </body>
    </html>
  );
}
```

**frontend/app/globals.css**

```
@import "tailwindcss";

:root {
  --background: #ffffff;
  --foreground: #1e293b;
  --primary: #2563eb;
  --primary-dark: #1e40af;
  --secondary: #64748b;
}

@theme inline {
  --color-background: var(--background);
  --color-foreground: var(--foreground);
  --color-primary: var(--primary);
  --color-primary-dark: var(--primary-dark);
  --color-secondary: var(--secondary);
  --font-sans: var(--font-geist-sans);
  --font-mono: var(--font-geist-mono);
}

body {
  background: var(--background);
  color: var(--foreground);
  font-family: Arial, Helvetica, sans-serif;
}

/* College-style blue/white theme */
.bg-primary {
  background-color: var(--primary);
}

.bg-primary-dark {
  background-color: var(--primary-dark);
}

.text-primary {
```

```
  color: var(--primary);
}
```

**frontend/app/login/page.tsx**

```
'use client';

import { useState, useEffect } from 'react';
import { useRouter } from 'next/navigation';
import Link from 'next/link';
import { authApi } from '@/lib/services';
import { useAuth } from '@/lib/auth-context';
import { toast } from 'sonner';

export default function LoginPage() {
  const router = useRouter();
  const { user, loading, refresh } = useAuth();
  const [formData, setFormData] = useState({ username: '', password: '' });
  const [submitting, setSubmitting] = useState(false);

  // If already logged in, redirect to dashboard
  useEffect(() => {
    if (!loading && user) {
      router.push('/dashboard');
    }
  }, [user, loading, router]);

  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault();
    setSubmitting(true);
    try {
      await authApi.login(formData.username, formData.password);
      // Refresh AuthContext so user state is populated BEFORE navigating
      await refresh();
      toast.success('Login successful!');
      router.push('/dashboard');
    } catch (error: any) {
      toast.error(error.response?.data?.error || 'Login failed');
    } finally {
      setSubmitting(false);
    }
  };

  return (
    <div className="min-h-screen flex items-center justify-center bg-gradient-to-br from-blue-50 to-
white">
      <div className="bg-white p-8 rounded-lg shadow-lg w-full max-w-md">
        <h1 className="text-3xl font-bold text-blue-900 mb-6 text-center">
          Login
        </h1>
        <form onSubmit={handleSubmit} className="space-y-4">
          <div>
            <label htmlFor="username" className="block text-sm font-medium text-gray-700 mb-1">
              Username
            </label>
            <input
              type="text"
              id="username"
              required
              value={formData.username}
              onChange={(e) => setFormData({ ...formData, username: e.target.value })}
              className="w-full px-4 py-2 border border-gray-300 rounded-lg focus:ring-2 focus:ring-blue-
500 focus:border-transparent"
            />
          </div>
          <div>
            <label htmlFor="password" className="block text-sm font-medium text-gray-700 mb-1">
              Password
            </label>
            <input
              type="password"
              id="password"
              required
              value={formData.password}
              onChange={(e) => setFormData({ ...formData, password: e.target.value })}
              className="w-full px-4 py-2 border border-gray-300 rounded-lg focus:ring-2 focus:ring-blue-
500 focus:border-transparent"
            />
```

```
          </div>
          <button
            type="submit"
            disabled={submitting}
            className="w-full bg-blue-600 text-white py-2 rounded-lg hover:bg-blue-700 transition-colors
disabled:opacity-50 disabled:cursor-not-allowed"
          >
            {submitting ? 'Logging in...' : 'Login'}
          </button>
        </form>
        <p className="mt-4 text-center text-sm text-gray-600">
          Don't have an account?{' '}
          <Link href="/register" className="text-blue-600 hover:underline">
            Contact Admin
          </Link>
        </p>
        <div className="mt-6 text-center">
          <Link href="/" className="text-gray-500 hover:text-blue-600 flex items-center justify-center
gap-2 group transition-colors">
            <span className="group-hover:-translate-x-1 transition-transform">←</span>
            Back to Home
          </Link>
        </div>
      </div>
    </div>
  );
}
```

**frontend/app/dashboard/page.tsx**

```
'use client';

import { useEffect, useState } from 'react';
import { useRouter } from 'next/navigation';
import { useAuth } from '@/lib/auth-context';
import { api } from '@/lib/api';
import { AvailabilitySlot } from '@/lib/types';
import { toast } from 'sonner';
import Link from 'next/link';
import { authApi } from '@/lib/services';

export default function DashboardPage() {
  const { user, loading, logout } = useAuth();
  const router = useRouter();

  // Availability state
  const [selectedDate, setSelectedDate] = useState(new Date().toISOString().split('T')[0]);
  const [availability, setAvailability] = useState<AvailabilitySlot[]>([]);
  const [checking, setChecking] = useState(false);
  const [pwdOpen, setPwdOpen] = useState(false);
  const [pwdCurrent, setPwdCurrent] = useState('');
  const [pwdNew, setPwdNew] = useState('');
  const [pwdConfirm, setPwdConfirm] = useState('');
  const [pwdSaving, setPwdSaving] = useState(false);

  useEffect(() => {
    if (!loading && !user) {
      router.replace('/login');
    }
  }, [user, loading, router]);

  const fetchAvailability = async () => {
    setChecking(true);
    try {
      const response = await api.get(`/availability?date=${selectedDate}`);
      setAvailability(response.data);
    } catch (error) {
      toast.error('Failed to fetch availability');
    } finally {
      setChecking(false);
    }
  };

  // Always show spinner while loading — never redirect prematurely
  if (loading) {
    return (
      <div className="min-h-screen flex items-center justify-center bg-gray-50">
```

```
        <div className="text-center">
          <div className="animate-spin rounded-full h-12 w-12 border-b-2 border-blue-600 mx-auto mb-4">
</div>
          <p className="text-gray-600">Loading...</p>
        </div>
      </div>
    );
  }

  if (!user) return null;

  const commonLinks = [
    { label: '🏛️ View Halls', href: '/halls' },
  ];

  const roleLinks: Record<string, { label: string; href: string }[]> = {
    admin: [
      { label: '📋 Pending Requests', href: '/admin/pending' },
      { label: '🗓️ All Bookings', href: '/admin/bookings' },
      { label: '🏛️ Manage Halls', href: '/admin/halls' },
      { label: '🏢 Manage Departments', href: '/admin/departments' },
      { label: '🎓 Manage Classes', href: '/admin/classes' },
      { label: '🏅 Manage Clubs', href: '/admin/clubs' },
      { label: '👥 Manage Users', href: '/admin/users' },
      { label: '📊 Reports', href: '/admin/reports' },
    ],
    dept_coordinator: [
      { label: '📋 Pending Requests', href: '/coordinator/pending' },
      { label: '🎓 Manage Classes', href: '/coordinator/classes' },
      { label: '👩‍🏫 View Faculty', href: '/coordinator/faculty' },
    ],
    requester: [
      { label: '➕ New Request', href: '/requester/request-new' },
      { label: '📋 My Requests', href: '/requester/my-requests' },
    ],
    faculty: [],
  };

  const links = [...commonLinks, ...(roleLinks[user.role] ?? [])];

  const handleOwnPasswordSubmit = async (e: React.FormEvent) => {
    e.preventDefault();
    if (pwdNew.length < 6) {
      toast.error('New password must be at least 6 characters');
      return;
    }
    if (pwdNew !== pwdConfirm) {
      toast.error('New passwords do not match');
      return;
    }
    setPwdSaving(true);
    try {
      await authApi.changeOwnPassword(pwdCurrent, pwdNew);
      toast.success('Password updated. Use it on your next login.');
      setPwdOpen(false);
      setPwdCurrent('');
      setPwdNew('');
      setPwdConfirm('');
    } catch (err: unknown) {
      const msg =
        err && typeof err === 'object' && 'response' in err
          ? (err as { response?: { data?: { error?: string } } }).response?.data?.error
          : undefined;
      toast.error(msg || 'Could not update password');
    } finally {
      setPwdSaving(false);
    }
  };

  const roleLabel: Record<string, string> = {
    admin: 'Administrator',
    dept_coordinator: 'Department Coordinator',
    requester: 'Requester',
    faculty: 'Faculty',
  };

  return (
```

```
    <div className="min-h-screen bg-gray-50 relative">
      {pwdOpen && (
        <div className="fixed inset-0 z-50 flex items-center justify-center p-4 bg-black/40" role="dialog"
aria-modal="true">
          <form
            onSubmit={handleOwnPasswordSubmit}
            className="bg-white rounded-xl shadow-xl border border-gray-200 w-full max-w-md p-6"
          >
            <h2 className="text-lg font-bold text-gray-900 mb-1">Change your password</h2>
            <p className="text-sm text-gray-600 mb-4">Enter your current password, then choose a new one.
</p>
            <div className="space-y-3 mb-5">
              <div>
                <label className="block text-sm font-medium text-gray-700 mb-1">Current password</label>
                <input
                  type="password"
                  autoComplete="current-password"
                  required
                  className="w-full border border-gray-300 rounded-lg px-3 py-2 text-sm focus:ring-2
focus:ring-blue-500"
                  value={pwdCurrent}
                  onChange={(e) => setPwdCurrent(e.target.value)}
                />
              </div>
              <div>
                <label className="block text-sm font-medium text-gray-700 mb-1">New password</label>
                <input
                  type="password"
                  autoComplete="new-password"
                  required
                  minLength={6}
                  className="w-full border border-gray-300 rounded-lg px-3 py-2 text-sm focus:ring-2
focus:ring-blue-500"
                  value={pwdNew}
                  onChange={(e) => setPwdNew(e.target.value)}
                />
              </div>
              <div>
                <label className="block text-sm font-medium text-gray-700 mb-1">Confirm new
password</label>
                <input
                  type="password"
                  autoComplete="new-password"
                  required
                  minLength={6}
                  className="w-full border border-gray-300 rounded-lg px-3 py-2 text-sm focus:ring-2
focus:ring-blue-500"
                  value={pwdConfirm}
                  onChange={(e) => setPwdConfirm(e.target.value)}
                />
              </div>
            </div>
            <div className="flex gap-3 justify-end">
              <button
                type="button"
                onClick={() => {
                  setPwdOpen(false);
                  setPwdCurrent('');
                  setPwdNew('');
                  setPwdConfirm('');
                }}
                className="px-4 py-2 rounded-lg text-sm bg-gray-100 hover:bg-gray-200 text-gray-700"
              >
                Cancel
              </button>
              <button
                type="submit"
                disabled={pwdSaving}
                className="px-4 py-2 rounded-lg text-sm bg-blue-600 hover:bg-blue-700 text-white
disabled:opacity-50"
              >
                {pwdSaving ? 'Saving…' : 'Update password'}
              </button>
            </div>
          </form>
        </div>
      )}
```

```
      {/* Header */}
      <header className="bg-blue-900 text-white px-6 py-4 flex items-center justify-between shadow-md">
        <div>
          <h1 className="text-xl font-bold">Seminar Hall Booking System</h1>
          <p className="text-blue-200 text-sm">{roleLabel[user.role]}</p>
        </div>
        <div className="flex items-center gap-4">
          <span className="text-sm text-blue-200">Welcome, {user.full_name}</span>
          <button
            onClick={logout}
            className="bg-blue-700 hover:bg-blue-600 text-white text-sm px-4 py-2 rounded-lg transition-
colors"
          >
            Logout
          </button>
        </div>
      </header>

      <div className="max-w-4xl mx-auto px-4 py-8">
        <div className="flex justify-between items-center mb-6">
          <h2 className="text-2xl font-bold text-gray-800">Dashboard</h2>
        </div>

        <div className="space-y-8">
          {/* Main Links - 3 columns on desktop for better spacing */}
          <div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-3 gap-4">
            {links.map((link) => (
              <a
                key={link.href}
                href={link.href}
                className="bg-white rounded-xl shadow-sm border border-gray-100 p-5 hover:shadow-md
hover:border-blue-300 transition-all group"
              >
                <p className="text-lg font-semibold text-gray-800 group-hover:text-blue-700 transition-
colors">
                  {link.label}
                </p>
              </a>
            ))}
          </div>

          {/* Availability Checker - Full Width Row */}
          <div className="bg-white p-6 rounded-xl shadow-sm border border-gray-100">
            <h3 className="text-xl font-bold text-gray-800 mb-6 flex items-center gap-2">
              <span>🗓️</span> Check Hall Availability
            </h3>
            <div className="flex flex-col md:flex-row gap-4 mb-6">
              <input
                type="date"
                value={selectedDate}
                onChange={(e) => setSelectedDate(e.target.value)}
                className="flex-1 px-4 py-3 border border-gray-300 rounded-lg focus:ring-2 focus:ring-
blue-500 text-base"
              />
              <button
                onClick={fetchAvailability}
                disabled={checking}
                className="bg-blue-600 text-white px-8 py-3 rounded-lg hover:bg-blue-700 transition-colors
disabled:opacity-50 font-bold whitespace-nowrap"
              >
                {checking ? 'Checking...' : 'Check Availability'}
              </button>
            </div>

            {availability.length > 0 && (
              <div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-3 gap-3 pt-4 border-t border-
gray-100">
                {availability.map((slot) => (
                  <div key={slot.hall_id} className="p-4 bg-blue-50 rounded-lg border border-blue-100 flex
flex-col justify-between">
                    <div>
                      <p className="font-bold text-blue-900">{slot.hall_name}</p>
                      <p className="text-xs text-gray-500 mb-3">
                        {(slot.bookings ?? []).length} booking(s) scheduled
                      </p>
                    </div>
```

```
                  <Link
                    href={`/halls/${slot.hall_id}/calendar`}
                    className="text-xs text-blue-600 hover:text-blue-800 font-bold uppercase tracking-
widest text-right mt-auto"
                  >
                    View Calendar →
                  </Link>
                </div>
              ))}
            </div>
          )}

          {availability.length === 0 && !checking && (
            <p className="text-sm text-gray-400 italic text-center py-6 bg-gray-50 rounded-lg border
border-dashed border-gray-200">
              Pick a date above to see which halls are booked or free.
            </p>
          )}
        </div>

        {/* User Profile Info */}
        <div className="bg-white rounded-xl shadow-sm border border-gray-100 p-6">
          <div className="flex flex-wrap items-center justify-between gap-3 mb-4 pb-2 border-b border-
gray-50">
            <h3 className="text-lg font-semibold text-gray-700">Your Profile</h3>
            <button
              type="button"
              onClick={() => {
                setPwdOpen(true);
                setPwdCurrent('');
                setPwdNew('');
                setPwdConfirm('');
              }}
              className="text-sm font-medium text-blue-600 hover:text-blue-800 px-3 py-1.5 rounded-lg
border border-blue-200 hover:bg-blue-50 transition-colors"
            >
              Change password
            </button>
          </div>
          <div className="grid grid-cols-1 sm:grid-cols-2 gap-y-4 gap-x-8 text-sm">
            <div className="flex flex-col">
              <span className="text-gray-400 text-xs font-medium uppercase tracking-wider mb-0.5">Full
Name</span>
              <span className="text-gray-800 font-semibold">{user.full_name}</span>
            </div>
            <div className="flex flex-col">
              <span className="text-gray-400 text-xs font-medium uppercase tracking-wider mb-
0.5">Username</span>
              <span className="text-gray-800 font-semibold">{user.username}</span>
            </div>
            <div className="flex flex-col">
              <span className="text-gray-400 text-xs font-medium uppercase tracking-wider mb-0.5">Email
Address</span>
              <span className="text-gray-800 font-semibold">{user.email ?? 'Not provided'}</span>
            </div>
            <div className="flex flex-col">
              <span className="text-gray-400 text-xs font-medium uppercase tracking-wider mb-
0.5">Account Role</span>
              <span className="text-blue-700 font-bold">{roleLabel[user.role]}</span>
            </div>
          </div>
        </div>
      </div>
    </div>
  );
}
```

**frontend/app/halls/page.tsx**

```
'use client';

import { useEffect, useState } from 'react';
import Link from 'next/link';
import { useRouter } from 'next/navigation';
import { api } from '@/lib/api';
import { Hall } from '@/lib/types';
import { toast } from 'sonner';
```

```
export default function HallsPage() {
  const router = useRouter();
  const [halls, setHalls] = useState<Hall[]>([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    fetchHalls();
  }, []);

  const fetchHalls = async () => {
    try {
      const response = await api.get('/halls');
      setHalls(response.data);
    } catch (error) {
      toast.error('Failed to fetch halls');
    } finally {
      setLoading(false);
    }
  };

  if (loading) {
    return (
      <div className="min-h-screen flex items-center justify-center">
        <div className="text-xl">Loading...</div>
      </div>
    );
  }

  return (
    <div className="min-h-screen bg-gradient-to-br from-blue-50 to-white">
      <div className="container mx-auto px-4 py-8">
        <div className="flex justify-between items-center mb-8">
          <h1 className="text-4xl font-bold text-blue-900">Seminar Halls</h1>
          <button
            onClick={() => router.back()}
            className="text-blue-600 hover:underline font-medium"
          >
            ← Back
          </button>
        </div>

        <div className="grid md:grid-cols-2 lg:grid-cols-3 gap-6">
          {halls.map((hall) => (
            <div
              key={hall.hall_id}
              className="bg-white p-6 rounded-lg shadow-md hover:shadow-lg transition-shadow"
            >
              <h2 className="text-2xl font-semibold text-blue-900 mb-2">
                {hall.hall_name}
              </h2>
              <p className="text-gray-600 mb-4">
                <strong>Capacity:</strong> {hall.capacity} people
              </p>
              {hall.location && (
                <p className="text-gray-600 mb-2">
                  <strong>Location:</strong> {hall.location}
                </p>
              )}
              <div className="flex flex-wrap gap-1 mb-4">
                {Object.entries(hall.facilities || {}).map(([f, v]) => v && (
                  <span key={f} className="text-[10px] bg-blue-50 text-blue-700 px-1.5 py-0.5 rounded
border border-blue-100 uppercase">
                    {f.replace('_', ' ')}
                  </span>
                ))}
              </div>
              <Link
                href={`/halls/${hall.hall_id}/calendar`}
                className="text-blue-600 hover:underline inline-block font-medium"
              >
                View Availability →
              </Link>
            </div>
          ))}
        </div>
      </div>
```

```
    </div>
  );
}
```

**frontend/app/halls/[id]/calendar/page.tsx**

```
'use client';

import { useState, useEffect } from 'react';
import { useParams, useRouter } from 'next/navigation';
import Link from 'next/link';
import { Calendar, momentLocalizer, View } from 'react-big-calendar';
import moment from 'moment';
import 'react-big-calendar/lib/css/react-big-calendar.css';
import { api } from '@/lib/api';
import { Booking, Hall } from '@/lib/types';
import { formatTime } from '@/lib/utils';
import { toast } from 'sonner';

const localizer = momentLocalizer(moment);

export default function HallCalendarPage() {
  const params = useParams();
  const router = useRouter();
  const hallId = params.id as string;
  const [hall, setHall] = useState<Hall | null>(null);
  const [bookings, setBookings] = useState<Booking[]>([]);
  const [loading, setLoading] = useState(true);
  const [selectedDate, setSelectedDate] = useState(new Date());
  const [view, setView] = useState<View>('day');

  useEffect(() => {
    fetchHall();
  }, [hallId]);

  useEffect(() => {
    fetchBookings();
  }, [hallId, selectedDate]);

  const fetchHall = async () => {
    if (!hallId) return;
    try {
      const response = await api.get(`/halls/${hallId}`);
      setHall(response.data);
    } catch (error: any) {
      toast.error('Failed to fetch hall details');
    }
  };

  const fetchBookings = async () => {
    if (!hallId) return;
    try {
      // Fetch for the whole month surrounding selectedDate to be safe regardless of view
      const startOfMonth = moment(selectedDate).startOf('month').format('YYYY-MM-DD');
      const endOfMonth = moment(selectedDate).endOf('month').format('YYYY-MM-DD');

      const response = await api.get(`/availability?
start_date=${startOfMonth}&end_date=${endOfMonth}&hall_id=${hallId}`);
      setBookings(response.data || []);
    } catch (error) {
      toast.error('Failed to fetch bookings');
    } finally {
      setLoading(false);
    }
  };

  const events = (bookings || []).map((booking) => {
    const datePart = booking.event_date.split('T')[0];
    return {
      title: booking.event_title,
      start: moment(`${datePart} ${booking.start_time}`).toDate(),
      end: moment(`${datePart} ${booking.end_time}`).toDate(),
      resource: booking,
    };
  });

  if (loading) {
```

```
    return (
      <div className="min-h-screen flex items-center justify-center">
        <div className="text-xl">Loading...</div>
      </div>
    );
  }

  return (
    <div className="min-h-screen bg-gradient-to-br from-blue-50 to-white">
      <div className="container mx-auto px-4 py-8">
        <div className="flex items-center justify-between mb-6">
          <div>
            <h1 className="text-3xl font-bold text-blue-900">
              {hall?.hall_name || 'Hall Calendar'}
            </h1>
            {hall && (
              <p className="text-gray-600 mt-1">
                Capacity: {hall.capacity} | {hall.location}
              </p>
            )}
          </div>
          <button
            onClick={() => router.back()}
            className="text-blue-600 hover:underline flex items-center gap-1 font-medium"
          >
            ← Back
          </button>
        </div>

        <div className="bg-white p-4 rounded-lg shadow-md mb-6">
          <label className="block text-sm font-medium text-gray-700 mb-2">
            Select Date
          </label>
          <input
            type="date"
            value={moment(selectedDate).format('YYYY-MM-DD')}
            onChange={(e) => setSelectedDate(moment(e.target.value).toDate())}
            className="px-4 py-2 border border-gray-300 rounded-lg focus:ring-2 focus:ring-blue-500
focus:border-transparent"
          />
        </div>

        <div className="bg-white p-6 rounded-lg shadow-md" style={{ height: '600px' }}>
          <Calendar
            localizer={localizer}
            events={events}
            startAccessor="start"
            endAccessor="end"
            style={{ height: '100%' }}
            view={view}
            onView={(newView) => setView(newView)}
            views={['day', 'week', 'month']}
            date={selectedDate}
            onNavigate={(date) => setSelectedDate(date)}
          />
        </div>

        {(bookings || []).length > 0 && view === 'day' && (
          <div className="mt-6 bg-white p-6 rounded-lg shadow-md">
            <h2 className="text-xl font-bold text-blue-900 mb-4">Bookings for
{selectedDate.toLocaleDateString()}</h2>
            <div className="space-y-2">
              {(bookings || []).filter(b => b.event_date.split('T')[0] ===
moment(selectedDate).format('YYYY-MM-DD')).map((booking) => (
                <div key={booking.booking_id} className="p-3 bg-blue-50 rounded">
                  <p className="font-semibold">{booking.event_title}</p>
                  <p className="text-sm text-gray-600">
                    {formatTime(booking.start_time)} - {formatTime(booking.end_time)}
                  </p>
                </div>
              ))}
            </div>
          </div>
        )}
      </div>
    </div>
  );
```

```
}
```

**frontend/app/requester/request-new/page.tsx**

```
'use client';

import { useEffect, useState } from 'react';
import { useRouter } from 'next/navigation';
import { useAuth } from '@/lib/auth-context';
import { hallsApi, requestsApi } from '@/lib/services';
import { Hall } from '@/lib/types';
import { toast } from 'sonner';

export default function NewRequestPage() {
  const { user, loading } = useAuth();
  const router = useRouter();
  const [halls, setHalls] = useState<Hall[]>([]);
  const [submitting, setSubmitting] = useState(false);

  const [form, setForm] = useState({
    hall_id: 0,
    event_title: '',
    event_description: '',
    event_date: '',
    start_time: '',
    end_time: '',
    expected_attendees: '',
    purpose: '',
  });

  useEffect(() => {
    if (!loading && !user) router.push('/login');
    if (!loading && user && user.role !== 'requester') router.push('/dashboard');
  }, [user, loading, router]);

  useEffect(() => {
    hallsApi.getAll().then((res) => setHalls(res.data)).catch(() => toast.error('Failed to load halls'));
  }, []);

  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault();
    if (!form.hall_id) { toast.error('Please select a hall'); return; }

    setSubmitting(true);
    try {
      await requestsApi.create({
        hall_id: form.hall_id,
        event_title: form.event_title,
        event_description: form.event_description || undefined,
        event_date: form.event_date,
        start_time: form.start_time,
        end_time: form.end_time,
        expected_attendees: form.expected_attendees ? parseInt(form.expected_attendees) : undefined,
        purpose: form.purpose || undefined,
      });
      toast.success('Request submitted successfully!');
      router.push('/requester/my-requests');
    } catch (err: any) {
      toast.error(err.response?.data?.error || 'Failed to submit request');
    } finally {
      setSubmitting(false);
    }
  };

  if (loading) return <div className="min-h-screen flex items-center justify-center"><div
className="animate-spin rounded-full h-12 w-12 border-b-2 border-blue-600"></div></div>;

  return (
    <div className="min-h-screen bg-gray-50">
      <header className="bg-blue-900 text-white px-6 py-4 flex items-center justify-between">
        <h1 className="text-xl font-bold">New Booking Request</h1>
        <button onClick={() => router.push('/dashboard')} className="bg-blue-800 hover:bg-blue-700 px-4
py-2 rounded-lg text-sm">Dashboard</button>
      </header>

      <div className="max-w-2xl mx-auto px-4 py-8">
        <form onSubmit={handleSubmit} className="bg-white rounded-xl shadow-sm border border-gray-100 p-6
```

```
space-y-5">
          <div>
            <label className="block text-sm font-medium text-gray-700 mb-1">Seminar Hall *</label>
            <select
              required
              className="w-full border border-gray-300 rounded-lg px-3 py-2 focus:ring-2 focus:ring-blue-
500"
              value={form.hall_id}
              onChange={(e) => setForm({ ...form, hall_id: parseInt(e.target.value) })}
            >
              <option value={0}>Select a hall...</option>
              {halls.map((h) => (
                <option key={h.hall_id} value={h.hall_id}>{h.hall_name} (Capacity: {h.capacity})</option>
              ))}
            </select>
          </div>

          <div>
            <label className="block text-sm font-medium text-gray-700 mb-1">Event Title *</label>
            <input required type="text" className="w-full border border-gray-300 rounded-lg px-3 py-2
focus:ring-2 focus:ring-blue-500"
              value={form.event_title} onChange={(e) => setForm({ ...form, event_title: e.target.value })}
/>
          </div>

          <div>
            <label className="block text-sm font-medium text-gray-700 mb-1">Event Date *</label>
            <input required type="date" className="w-full border border-gray-300 rounded-lg px-3 py-2
focus:ring-2 focus:ring-blue-500"
              value={form.event_date} onChange={(e) => setForm({ ...form, event_date: e.target.value })}
/>
          </div>

          <div className="grid grid-cols-2 gap-4">
            <div>
              <label className="block text-sm font-medium text-gray-700 mb-1">Start Time *</label>
              <input required type="time" className="w-full border border-gray-300 rounded-lg px-3 py-2
focus:ring-2 focus:ring-blue-500"
                value={form.start_time} onChange={(e) => setForm({ ...form, start_time: e.target.value })}
/>
            </div>
            <div>
              <label className="block text-sm font-medium text-gray-700 mb-1">End Time *</label>
              <input required type="time" className="w-full border border-gray-300 rounded-lg px-3 py-2
focus:ring-2 focus:ring-blue-500"
                value={form.end_time} onChange={(e) => setForm({ ...form, end_time: e.target.value })} />
            </div>
          </div>

          <div>
            <label className="block text-sm font-medium text-gray-700 mb-1">Expected Attendees</label>
            <input type="number" min={1} className="w-full border border-gray-300 rounded-lg px-3 py-2
focus:ring-2 focus:ring-blue-500"
              value={form.expected_attendees} onChange={(e) => setForm({ ...form, expected_attendees:
e.target.value })} />
          </div>

          <div>
            <label className="block text-sm font-medium text-gray-700 mb-1">Purpose</label>
            <input type="text" className="w-full border border-gray-300 rounded-lg px-3 py-2 focus:ring-2
focus:ring-blue-500"
              value={form.purpose} onChange={(e) => setForm({ ...form, purpose: e.target.value })} />
          </div>

          <div>
            <label className="block text-sm font-medium text-gray-700 mb-1">Event Description</label>
            <textarea rows={3} className="w-full border border-gray-300 rounded-lg px-3 py-2 focus:ring-2
focus:ring-blue-500"
              value={form.event_description} onChange={(e) => setForm({ ...form, event_description:
e.target.value })} />
          </div>

          <button type="submit" disabled={submitting}
            className="w-full bg-blue-600 text-white py-2.5 rounded-lg hover:bg-blue-700 transition-colors
disabled:opacity-50 font-medium">
            {submitting ? 'Submitting...' : 'Submit Request'}
          </button>
```

```
        </form>
      </div>
    </div>
  );
}
```

**frontend/app/requester/my-requests/page.tsx**

```
'use client';

import { useEffect, useState } from 'react';
import { useRouter } from 'next/navigation';
import { useAuth } from '@/lib/auth-context';
import { requestsApi } from '@/lib/services';
import { Request } from '@/lib/types';
import { formatDate, formatTime } from '@/lib/utils';
import { toast } from 'sonner';

export default function MyRequestsPage() {
  const { user, loading } = useAuth();
  const router = useRouter();
  const [requests, setRequests] = useState<Request[]>([]);
  const [fetching, setFetching] = useState(true);

  useEffect(() => {
    if (!loading && !user) router.push('/login');
    if (!loading && user && user.role !== 'requester') router.push('/dashboard');
  }, [user, loading, router]);

  useEffect(() => {
    if (user?.role === 'requester') {
      requestsApi.getMy()
        .then((res) => setRequests(res.data))
        .catch(() => toast.error('Failed to load requests'))
        .finally(() => setFetching(false));
    }
  }, [user]);

  const statusColor: Record<string, string> = {
    pending: 'bg-yellow-100 text-yellow-800',
    forwarded: 'bg-blue-100 text-blue-800',
    approved: 'bg-green-100 text-green-800',
    rejected: 'bg-red-100 text-red-800',
    amended: 'bg-purple-100 text-purple-800',
  };

  if (loading || fetching) return (
    <div className="min-h-screen flex items-center justify-center">
      <div className="animate-spin rounded-full h-12 w-12 border-b-2 border-blue-600"></div>
    </div>
  );

  return (
    <div className="min-h-screen bg-gray-50">
      <header className="bg-blue-900 text-white px-6 py-4 flex items-center justify-between">
        <h1 className="text-xl font-bold">My Booking Requests</h1>
        <div className="flex gap-3">
          <button onClick={() => router.push('/requester/request-new')} className="bg-blue-600 hover:bg-
blue-500 text-white px-4 py-2 rounded-lg text-sm transition-colors">
            + New Request
          </button>
          <button onClick={() => router.push('/dashboard')} className="bg-blue-800 hover:bg-blue-700 px-4
py-2 rounded-lg text-sm transition-colors">
            Dashboard
          </button>
        </div>
      </header>

      <div className="max-w-5xl mx-auto px-4 py-8">
        {requests.length === 0 ? (
          <div className="text-center py-16 text-gray-500">
            <p className="text-lg">No requests yet.</p>
            <button onClick={() => router.push('/requester/request-new')} className="mt-4 bg-blue-600
text-white px-6 py-2 rounded-lg hover:bg-blue-700">
              Make Your First Request
            </button>
          </div>
        ) : (
```

```
          <div className="space-y-4">
            {requests.map((req) => (
              <div key={req.request_id} className="bg-white rounded-xl shadow-sm border border-gray-100 p-
5">
                <div className="flex items-start justify-between">
                  <div>
                    <h3 className="font-semibold text-gray-800 text-lg">{req.event_title}</h3>
                    <p className="text-sm text-gray-500">{formatDate(req.event_date)} &bull;
{formatTime(req.start_time)} – {formatTime(req.end_time)}</p>
                    <p className="text-sm text-gray-600 mt-1">
                      Hall: <span className="font-medium">{req.hall_name ?? `Hall #${req.hall_id}`}</span>
                      {req.hall_location ? ` (${req.hall_location})` : ''}
                      {req.hall_capacity ? ` • Capacity: ${req.hall_capacity}` : ''}
                    </p>
                    {req.purpose && <p className="text-sm text-gray-600 mt-1">Purpose: {req.purpose}</p>}
                  </div>
                  <div className="flex flex-col items-end gap-2 text-xs font-medium">
                    <span className={`px-2 py-1 rounded-full ${statusColor[req.dept_status]}`}>Dept:
{req.dept_status}</span>
                    <span className={`px-2 py-1 rounded-full ${statusColor[req.admin_status]}`}>Admin:
{req.admin_status}</span>
                  </div>
                </div>
                {(req.dept_remarks || req.admin_remarks) && (
                  <div className="mt-3 border-t pt-3 text-sm text-gray-500 space-y-1">
                    {req.dept_remarks && <p>Dept remarks: {req.dept_remarks}</p>}
                    {req.admin_remarks && <p>Admin remarks: {req.admin_remarks}</p>}
                  </div>
                )}
              </div>
            ))}
          </div>
        )}
      </div>
    </div>
  );
}
```

**frontend/app/coordinator/pending/page.tsx**

```
'use client';

import { useEffect, useState } from 'react';
import { useRouter } from 'next/navigation';
import { useAuth } from '@/lib/auth-context';
import { requestsApi } from '@/lib/services';
import { Request } from '@/lib/types';
import { formatDate, formatTime } from '@/lib/utils';
import { toast } from 'sonner';

export default function CoordinatorPendingPage() {
  const { user, loading } = useAuth();
  const router = useRouter();
  const [requests, setRequests] = useState<Request[]>([]);
  const [fetching, setFetching] = useState(true);
  const [reviewing, setReviewing] = useState<number | null>(null);

  useEffect(() => {
    if (!loading && !user) router.push('/login');
    if (!loading && user && user.role !== 'dept_coordinator') router.push('/dashboard');
  }, [user, loading, router]);

  const fetchRequests = () => {
    requestsApi.getDeptPending()
      .then((res) => setRequests(res.data))
      .catch(() => toast.error('Failed to load requests'))
      .finally(() => setFetching(false));
  };

  useEffect(() => {
    if (user?.role === 'dept_coordinator') fetchRequests();
  }, [user]);

  const handleReview = async (id: number, action: 'approve' | 'reject') => {
    const remarks = action === 'reject' ? window.prompt('Enter rejection remarks (optional):') ??
undefined : undefined;
    setReviewing(id);
    try {
```

```
      await requestsApi.deptReview(id, { action, remarks });
      toast.success(`Request ${action === 'approve' ? 'forwarded to admin' : 'rejected'}`);
      fetchRequests();
    } catch (err: any) {
      toast.error(err.response?.data?.error || 'Action failed');
    } finally {
      setReviewing(null);
    }
  };

  if (loading || fetching) return <div className="min-h-screen flex items-center justify-center"><div
className="animate-spin rounded-full h-12 w-12 border-b-2 border-blue-600"></div></div>;

  return (
    <div className="min-h-screen bg-gray-50">
      <header className="bg-blue-900 text-white px-6 py-4 flex items-center justify-between">
        <h1 className="text-xl font-bold">Department Pending Requests</h1>
        <button onClick={() => router.push('/dashboard')} className="bg-blue-800 hover:bg-blue-700 px-4
py-2 rounded-lg text-sm">Dashboard</button>
      </header>

      <div className="max-w-5xl mx-auto px-4 py-8">
        {requests.length === 0 ? (
          <div className="text-center py-16 text-gray-500"><p className="text-lg">No pending requests for
your department.</p></div>
        ) : (
          <div className="space-y-4">
            {requests.map((req) => (
              <div key={req.request_id} className="bg-white rounded-xl shadow-sm border border-gray-100 p-
5">
                <div className="flex items-start justify-between">
                  <div>
                    <h3 className="font-semibold text-gray-800 text-lg">{req.event_title}</h3>
                    <p className="text-sm text-gray-500">{formatDate(req.event_date)} &bull;
{formatTime(req.start_time)} – {formatTime(req.end_time)}</p>
                    <p className="text-sm text-gray-600 mt-1">
                      Hall: <span className="font-medium">{req.hall_name ?? `Hall #${req.hall_id}`}</span>
                      {req.hall_location ? ` (${req.hall_location})` : ''}
                      {req.hall_capacity ? ` • Capacity: ${req.hall_capacity}` : ''}
                    </p>
                    {req.purpose && <p className="text-sm text-gray-600 mt-1">Purpose: {req.purpose}</p>}
                    {req.expected_attendees && <p className="text-sm text-gray-600">Attendees:
{req.expected_attendees}</p>}
                  </div>
                  <div className="flex gap-2">
                    <button
                      disabled={reviewing === req.request_id}
                      onClick={() => handleReview(req.request_id, 'approve')}
                      className="bg-green-600 hover:bg-green-700 text-white text-sm px-4 py-2 rounded-lg
transition-colors disabled:opacity-50"
                    >
                      ✓ Forward
                    </button>
                    <button
                      disabled={reviewing === req.request_id}
                      onClick={() => handleReview(req.request_id, 'reject')}
                      className="bg-red-600 hover:bg-red-700 text-white text-sm px-4 py-2 rounded-lg
transition-colors disabled:opacity-50"
                    >
                      ✗ Reject
                    </button>
                  </div>
                </div>
              </div>
            ))}
          </div>
        )}
      </div>
    </div>
  );
}
```

**frontend/lib/api.ts**

```
import axios from 'axios';

const API_URL = process.env.NEXT_PUBLIC_API_URL || 'http://localhost:8080';
```

```
export const api = axios.create({
  baseURL: `${API_URL}/api`,
  headers: {
    'Content-Type': 'application/json',
  },
});

// Attach JWT token from localStorage on every request
api.interceptors.request.use((config) => {
  if (typeof window !== 'undefined') {
    const token = localStorage.getItem('auth_token');
    if (token) {
      config.headers.Authorization = `Bearer ${token}`;
    }
  }
  return config;
});

// Response interceptor for error handling
api.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response?.status === 401) {
      // Only redirect if we're on a protected page (not on login or public pages)
      if (typeof window !== 'undefined') {
        const publicPaths = ['/login', '/', '/halls'];
        const isPublic = publicPaths.some((p) =>
          window.location.pathname === p || window.location.pathname.startsWith(`${p}/`)
        );
        if (!isPublic) {
          localStorage.removeItem('auth_token');
          window.location.href = '/login';
        }
      }
    }
    return Promise.reject(error);
  }
);
```

**frontend/lib/auth-context.tsx**

```
'use client';

import { createContext, useContext, useEffect, useState, ReactNode } from 'react';
import { authApi } from '@/lib/services';
import { User } from '@/lib/types';

interface AuthContextType {
  user: User | null;
  requesterInfo: { requester_id: number; requester_type: string } | null;
  loading: boolean;
  logout: () => Promise<void>;
  refresh: () => Promise<void>;
}

const AuthContext = createContext<AuthContextType | null>(null);

export function AuthProvider({ children }: { children: ReactNode }) {
  const [user, setUser] = useState<User | null>(null);
  const [requesterInfo, setRequesterInfo] = useState<{ requester_id: number; requester_type: string } |
null>(null);
  const [loading, setLoading] = useState(true);

  const refresh = async () => {
    try {
      const res = await authApi.getMe();
      setUser(res.data.user);
      setRequesterInfo(res.data.requester ?? null);
    } catch {
      setUser(null);
      setRequesterInfo(null);
    } finally {
      setLoading(false);
    }
  };
```

```
  const logout = async () => {
    try {
      await authApi.logout();
    } finally {
      localStorage.removeItem('auth_token');
      setUser(null);
      setRequesterInfo(null);
      window.location.href = '/login';
    }
  };

  useEffect(() => {
    refresh();
  }, []);

  return (
    <AuthContext.Provider value={{ user, requesterInfo, loading, logout, refresh }}>
      {children}
    </AuthContext.Provider>
  );
}

export function useAuth() {
  const ctx = useContext(AuthContext);
  if (!ctx) throw new Error('useAuth must be used inside AuthProvider');
  return ctx;
}
```

**frontend/lib/types.ts**

```
// User and Auth Types
export type UserRole = 'admin' | 'dept_coordinator' | 'requester' | 'faculty';
export type ApprovalStatus = 'pending' | 'forwarded' | 'approved' | 'rejected' | 'amended';
export type BookingStatus = 'confirmed' | 'cancelled' | 'completed' | 'no_show';
export type RequesterType = 'class' | 'club';

export interface User {
  user_id: number;
  username: string;
  full_name: string;
  email?: string;
  phone?: string;
  role: UserRole;
  dept_id?: number;
  is_active: boolean;
  created_at: string;
  updated_at: string;
}

export interface Requester {
  requester_id: number;
  user_id: number;
  requester_type: RequesterType;
  class_id?: number;
  club_id?: number;
  created_at: string;
}

export interface Hall {
  hall_id: number;
  hall_name: string;
  capacity: number;
  location?: string;
  facilities: Record<string, any>;
  is_active: boolean;
}

export interface Request {
  request_id: number;
  requester_id: number;
  hall_id: number;
  hall_name?: string;
  hall_location?: string;
  hall_capacity?: number;
  event_title: string;
  event_description?: string;
  event_date: string;
  start_time: string;
```

```
  end_time: string;
  expected_attendees?: number;
  purpose?: string;
  dept_status: ApprovalStatus;
  dept_approved_by?: number;
  dept_approved_at?: string;
  dept_remarks?: string;
  admin_status: ApprovalStatus;
  admin_approved_by?: number;
  admin_approved_at?: string;
  admin_remarks?: string;
  requested_at: string;
  is_cancelled: boolean;
}

export interface Booking {
  booking_id: number;
  request_id: number;
  hall_id: number;
  requester_id: number;
  event_title: string;
  event_date: string;
  start_time: string;
  end_time: string;
  status: BookingStatus;
  created_at: string;
  completed_at?: string;
}

export interface AvailabilitySlot {
  hall_id: number;
  hall_name: string;
  date: string;
  bookings: Booking[];
}

export interface LoginRequest {
  username: string;
  password: string;
}

export interface LoginResponse {
  user: User;
  token: string;
}

export interface CreateRequestInput {
  hall_id: number;
  event_title: string;
  event_description?: string;
  event_date: string;
  start_time: string;
  end_time: string;
  expected_attendees?: number;
  purpose?: string;
}

export interface ReviewRequestInput {
  action: 'approve' | 'reject';
  remarks?: string;
}

export interface Department {
  dept_id: number;
  dept_name: string;
  dept_code?: string;
  created_at: string;
}

export interface Class {
  class_id: number;
  class_name: string;
  dept_id: number;
  year: string;
}

export interface Club {
```

```
  club_id: number;
  club_name: string;
  dept_id?: number;
  description?: string;
}

export interface HallUsageReport {
  hall_id: number;
  hall_name: string;
  total_bookings: number;
}

export interface DepartmentUsageReport {
  dept_id: number;
  dept_name: string;
  total_requests: number;
}
```

**frontend/lib/services.ts**

```
import { api } from './api';
import {
  Hall, Booking, Department, Class, Club,
  HallUsageReport, DepartmentUsageReport,
  Request, User, ReviewRequestInput, CreateRequestInput,
} from './types';

// ─── AUTH ────────────────────────────────────────────────────────────────

export const authApi = {
  login: async (username: string, password: string) => {
    const res = await api.post<{ user: User; token: string }>('/auth/login', { username, password });
    // Save token to localStorage so the request interceptor can use it
    if (typeof window !== 'undefined') {
      localStorage.setItem('auth_token', res.data.token);
    }
    return res;
  },

  logout: () => api.post('/auth/logout'),

  getMe: () => api.get<{ user: User; requester?: { requester_id: number; requester_type: string } }>
('/auth/me'),

  /** Any logged-in user: verify current password and set a new one */
  changeOwnPassword: (currentPassword: string, newPassword: string) =>
    api.patch('/auth/password', {
      current_password: currentPassword,
      new_password: newPassword,
    }),
};

// ─── HALLS ───────────────────────────────────────────────────────────────

export const hallsApi = {
  getAll: () => api.get<Hall[]>('/halls'),

  getAvailability: (hallId: number, date: string) =>
    api.get<Booking[]>(`/availability?hall_id=${hallId}&date=${date}`),

  create: (data: Omit<Hall, 'hall_id' | 'is_active'>) => api.post<Hall>('/halls', data),
  update: (id: number, data: Partial<Hall>) => api.put<Hall>(`/halls/${id}`, data),
  delete: (id: number) => api.delete(`/halls/${id}`),
};

// ─── DEPARTMENTS ─────────────────────────────────────────────────────────

export const departmentsApi = {
  getAll: () => api.get<Department[]>('/departments'),
  create: (dept_name: string, dept_code: string) => api.post<Department>('/departments', { dept_name,
dept_code }),
  update: (id: number, dept_name: string, dept_code: string) => api.put(`/departments/${id}`, { dept_name,
dept_code }),
  delete: (id: number) => api.delete(`/departments/${id}`),
};

// ─── CLASSES ─────────────────────────────────────────────────────────────
```

```
export const classesApi = {
  getByDept: (deptId: number) => api.get<Class[]>(`/classes?dept_id=${deptId}`),
  create: (data: { class_name: string; dept_id: number; year: string }) => api.post<Class>('/classes',
data),
  delete: (id: number) => api.delete(`/classes/${id}`),
};

// ─── CLUBS ──────────────────────────────────────────────────────────

export const clubsApi = {
  getAll: (deptId?: number) => api.get<Club[]>(deptId ? `/clubs?dept_id=${deptId}` : '/clubs'),
  create: (data: { club_name: string; dept_id?: number; description?: string }) => api.post<Club>
('/clubs', data),
  delete: (id: number) => api.delete(`/clubs/${id}`),
};

// ─── REQUESTS ───────────────────────────────────────────────────────

export const requestsApi = {
  // Requester
  create: (data: CreateRequestInput) => api.post<Request>('/requests', data),
  getMy: () => api.get<Request[]>('/requests/my'),

  // Coordinator
  getDeptPending: () => api.get<Request[]>('/requests/dept-pending'),
  deptReview: (id: number, payload: ReviewRequestInput) => api.patch(`/requests/${id}/dept-review`,
payload),

  // Admin
  getAdminPending: () => api.get<Request[]>('/requests/admin-pending'),
  adminReview: (id: number, payload: ReviewRequestInput) => api.patch(`/requests/${id}/admin-review`,
payload),
};

// ─── BOOKINGS ───────────────────────────────────────────────────────

export const bookingsApi = {
  getAll: () => api.get<Booking[]>('/bookings'),
  cancel: (id: number) => api.patch(`/bookings/${id}/cancel`),
};

// ─── USERS ──────────────────────────────────────────────────────────

export const usersApi = {
  getAll: (role?: string) => api.get<User[]>(role ? `/users?role=${role}` : '/users'),
  getFaculties: () => api.get<User[]>('/users/faculties'),
  deactivate: (id: number) => api.delete(`/users/${id}`),
  /** Admin: set a new password for any user */
  changePassword: (userId: number, newPassword: string) =>
    api.patch(`/users/${userId}/password`, { new_password: newPassword }),
};

// ─── REPORTS ────────────────────────────────────────────────────────

export const reportsApi = {
  hallUsage: () => api.get<HallUsageReport[]>('/reports/hall-usage'),
  deptUsage: () => api.get<DepartmentUsageReport[]>('/reports/department-usage'),
};
```

**frontend/components/Navbar.tsx**

```
'use client';

import Link from 'next/link';
import { usePathname } from 'next/navigation';

export default function Navbar() {
  const pathname = usePathname();

  return (
    <nav className="bg-blue-900 text-white shadow-lg">
      <div className="container mx-auto px-4">
        <div className="flex justify-between items-center py-4">
          <Link href="/" className="text-2xl font-bold">
            Seminar Hall Booking
          </Link>
          <div className="flex gap-6">
            <Link
```

```
              href="/"
              className={`hover:text-blue-200 ${pathname === '/' ? 'underline' : ''}`}
            >
              Home
            </Link>
            <Link
              href="/halls"
              className={`hover:text-blue-200 ${pathname === '/halls' ? 'underline' : ''}`}
            >
              Halls
            </Link>
            <Link
              href="/dashboard"
              className={`hover:text-blue-200 ${pathname === '/dashboard' ? 'underline' : ''}`}
            >
              Dashboard
            </Link>
            <Link
              href="/login"
              className="bg-blue-700 px-4 py-2 rounded hover:bg-blue-600"
            >
              Login
            </Link>
          </div>
        </div>
      </div>
    </nav>
  );
}
```

**frontend/next.config.ts**

```
import type { NextConfig } from "next";

const nextConfig: NextConfig = {
  /* config options here */
};

export default nextConfig;
```

**frontend/tsconfig.json**

```
{
  "compilerOptions": {
    "target": "ES2017",
    "lib": ["dom", "dom.iterable", "esnext"],
    "allowJs": true,
    "skipLibCheck": true,
    "strict": true,
    "noEmit": true,
    "esModuleInterop": true,
    "module": "esnext",
    "moduleResolution": "bundler",
    "resolveJsonModule": true,
    "isolatedModules": true,
    "jsx": "react-jsx",
    "incremental": true,
    "plugins": [
      {
        "name": "next"
      }
    ],
    "paths": {
      "@/*": ["./*"]
    }
  },
  "include": [
    "next-env.d.ts",
    "**/*.ts",
    "**/*.tsx",
    ".next/types/**/*.ts",
    ".next/dev/types/**/*.ts",
    "**/*.mts"
  ],
  "exclude": ["node_modules"]
}
```

**frontend/package.json**

```
{
  "name": "frontend",
  "version": "0.1.0",
  "private": true,
  "scripts": {
    "dev": "next dev",
    "build": "next build",
    "start": "next start",
    "lint": "eslint"
  },
  "dependencies": {
    "@hookform/resolvers": "^5.2.2",
    "@types/moment": "^2.11.29",
    "@types/react-big-calendar": "^1.16.3",
    "axios": "^1.13.2",
    "date-fns": "^4.1.0",
    "moment": "^2.30.1",
    "next": "16.0.10",
    "react": "19.2.0",
    "react-big-calendar": "^1.19.4",
    "react-dom": "19.2.0",
    "react-hook-form": "^7.66.1",
    "sonner": "^2.0.7",
    "zod": "^4.1.13"
  },
  "devDependencies": {
    "@tailwindcss/postcss": "^4",
    "@types/node": "^20",
    "@types/react": "^19",
    "@types/react-dom": "^19",
    "eslint": "^9",
    "eslint-config-next": "16.0.5",
    "tailwindcss": "^4",
    "typescript": "^5"
  }
}
```